BHARATFORGE

End Term Report

Team 86

INDEX

1.	Problem Statement	02
2.	Our Approach	02
3.	Frontier Exploration	03
4.	Map Merging	03
5.	Object Detection	04
6.	Pathfinding	04
7.	Task Allocation	05
8.	LLM Integration	07
9.	Scalability Analysis	08
10.	Challenges Faced	09
11.	Future Scope and Improvements	10
12.	**Appendix**	

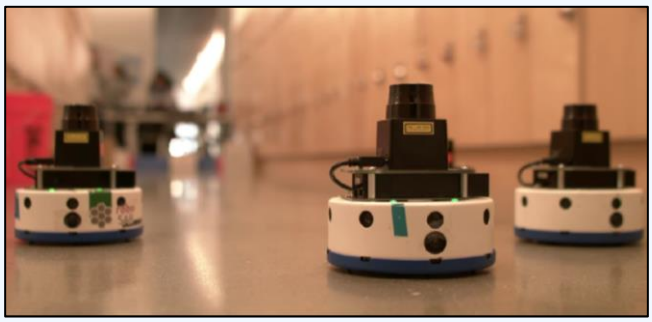
PROBLEM STATEMENT

"Software is the language of automation."

- Jensen Huang

Swarm robotics is increasingly being utilized in solving large-scale problems. It aims to bring about efficient collaboration between multiple autonomous agents to finish a set of tasks. However, many issues persist which hinder widespread adoption of this technique.

These problems are evident in dynamic indoor environments, especially if we rely on GPS or environmental interventions like ArUco markers. Such areas have moving obstacles, which interfere with robot task execution.



A multi-robot swarm system

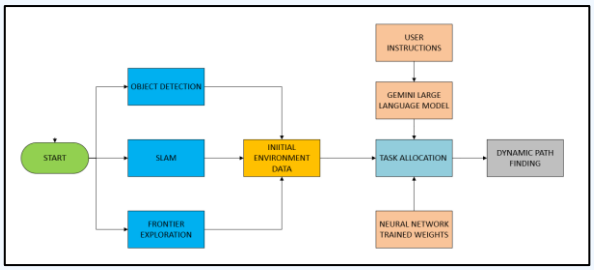
Therefore, we need to come up with novel systems which do not rely on traditional methods and instead use onboard sensor data for navigation, planning and task execution.

Such a system also must be robust and user-friendly, to facilitate widespread usage with minimal integration issues. One must also remember to make their design scalable, so that it can work for any number of robots.

OUR APPROACH

To tackle this problem, we have implemented the following systems

- Gmapping SLAM with map merging for multiple robots
- Greedy Frontier exploration for initial environment mapping
- YOLO based object detection along with object position estimation
- LLM and ML enhanced task allocation system.
- D* algorithm for path replanning and dynamic path avoidance
- Scalability features to extend this for any number of robots and tasks



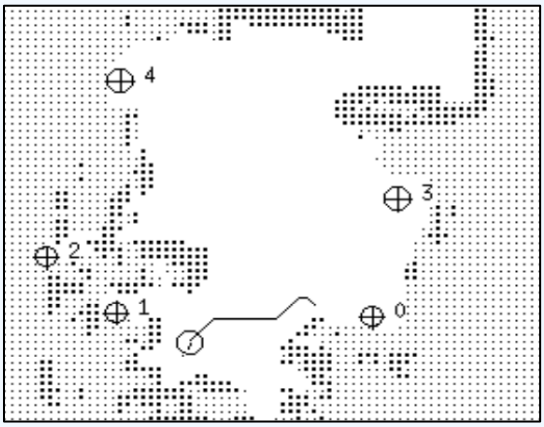
Overview of the entire process

We have first described the various mechanisms and algorithms while also analysing their performance. We then proceed to talk about how our system scales for multiple robots and tasks. We also discuss about the challenges we faced and areas of improvement

The appendix contains images of our own simulations of the various features which have been described.

FRONTIER EXPLORATION

Exploration of an unknown environment is a fundamental problem within the field of autonomous robots to create an initial state of the environment. Earlier methods such as wall following only work well in certain restrictive environments. To address these issues, a frontier exploration method was proposed.



Marking of frontiers [1]

The fundamental goal in initial map generation is gaining as much new information about the environment as possible. To maximize this, we move to the boundary between known areas and uncharted territory. We define these areas as “frontiers”.

By moving to successive frontiers, the robot can constantly increase its knowledge of the world. After exploring enough frontiers, our robots can generate a basic map which can be used for pathfinding in actual tasks.

In our approach we run frontier exploration for each robot, so that they can collaborate while creating a basic map of the environment. Frontier exploration is implemented using the explore-lite package. This package uses a greedy approach to explore all possible frontiers till we get a decent idea about our environment. This package also writes to the move_base package which controls robot movement.

MAP MERGING

While we are exploring an unknown area the robots must know their own position while also updating their own map of the environment. However, the indoor nature of our environment means that we cannot use traditional methods such as GPS or markers to do this. We can overcome this hurdle using Simultaneous Localization and Mapping (SLAM) methods instead.

A SLAM simulation in progress

There are many algorithms which can be used for SLAM. It is very popular as it creates a map of the environment and tells us the location of the robot simultaneously.

A LIDAR sensor

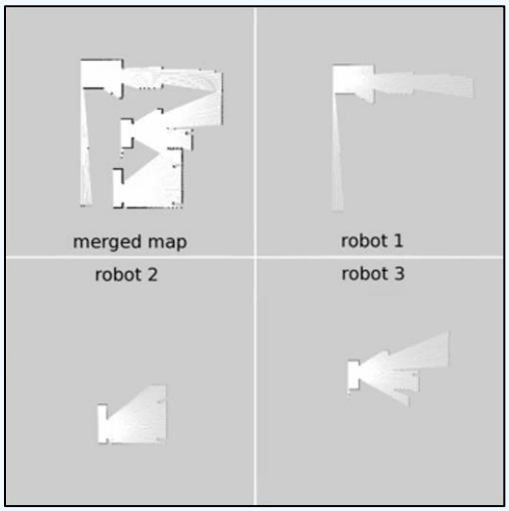
A popular way to achieve SLAM is to use LIDAR data. LIDAR (Light Detection and Ranging). Sensors emit laser in all directions and compute the time taken for the reflected light to return to the

[1] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in Proc. IEEE Int. Symp. Comput. Intell. Robot. Autom., 1997.

sensor. We can use this to compute the distance of the sensor from all angles of the environment.

To convert this LIDAR data to an actual mapping, we use the gmapping ROS package. This package generates a map for us and uses particle filters to estimate robot position.

The gmapping package generates a map for each individual robot. To combine these maps, we use the multirobot_map_merge ROS package. This package can be used for any number of robots.



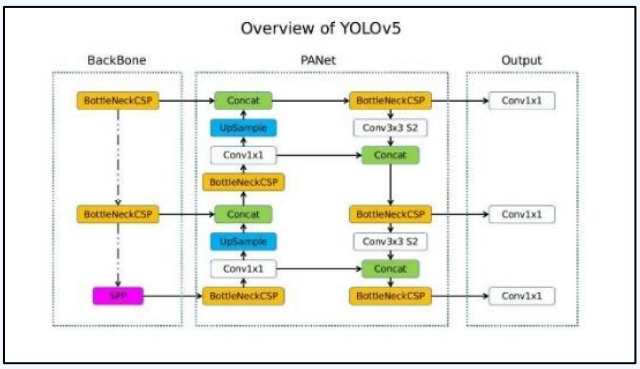
Map merging simulation[2]

The multirobot_map_merge package is very fast and runs in real time, giving better performance than other approaches.

OBJECT DETECTION

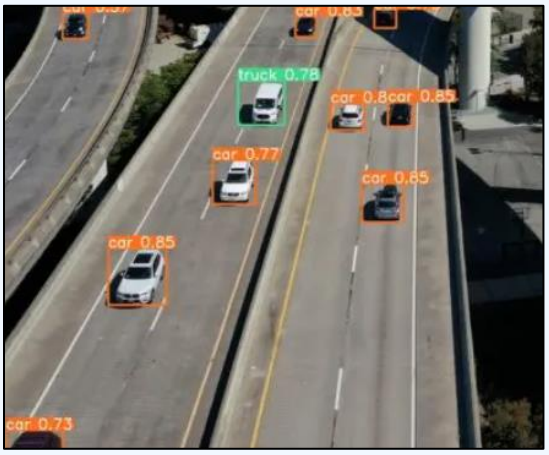
While exploring an environment, it is useful to note down the positions of various items. This can be useful later if the robot must move to these items later.

To carry out object detection we use the famous YOLOv5 model. YOLOv5 is a framework built using pytorch and is trained on millions of parameters which to detect objects inside an image. It uses Test-Time Augmentation (TTA) and Model Ensembling to make fast, accurate predictions.



An overview of the YOLOv5 architecture

At the heart of YOLOv5's architecture are Cross Stage Partial Networks (CSPNet). These networks split the base layer's feature map into two parts, then merge them through a cross-stage hierarchy. This decreases the complexity of the network while still retaining accuracy.



YOLOv5 model prediction

PATHFINDING

Finding the best path to the provided goals, although rather simple for a single robot, can become much more complicated for multiple robots in a single map. A classical approach would be to repeatedly apply a best algorithm like A* on the map after every map change. However, this is very computationally expensive, so we are compelled to use better methods which are more suited for dynamic environments.

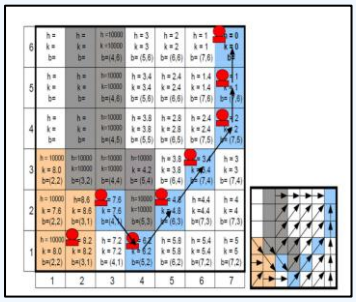
[2] http://wiki.ros.org/multirobot_map_merge

D* Algorithm

D star, or Dynamic A star, is an algorithm specially designed for maps with dynamic obstacles. In this algorithm, we perform a graph search for the initial path and then simply update the nodes instead of updating the entire graph if we encounter a dynamic obstacle.

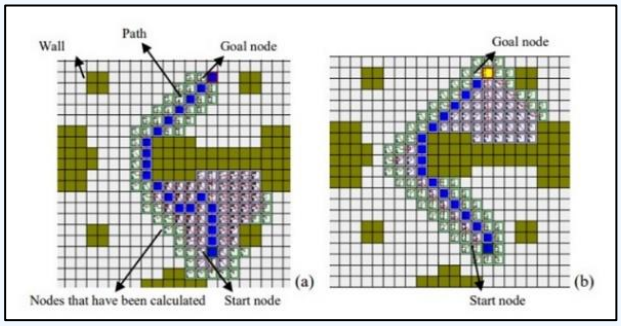
Unlike LPA*(Lifelong Planning A*), which checks the path from start to goal at every update, the D* algorithm reverses this search and updates from the goal to the start. It finds an initial optimal map and recalculates path whenever a new obstacle is encountered.

D* profits from this approach as it reduces this calculation even further by only updating from the goal to the current position when an obstacle is encountered. This makes the D* algorithm one of the best methods of handling dynamically changing maps and obstacles.



An overview of the D* algorithm

Performance Analysis



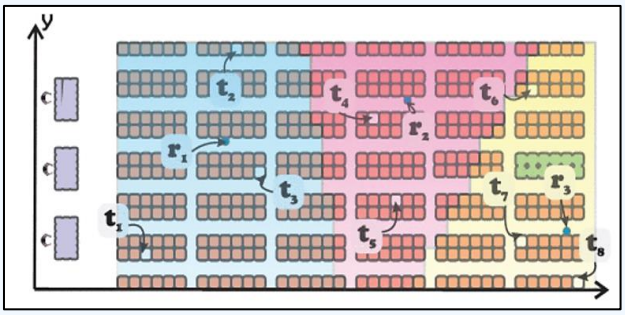
Simulations of a) A* algorithm and b) D* algorithm

The D* algorithm does not rebuild the path every single time an obstacle is detected. Hence it is more efficient in dynamic environments like indoor spaces.

TASK ALLOCATION

For our robots to collaborate effectively, we must ensure that all the tasks are divided in an optimal manner. While naïve implementations do exist, these do not consider the layout of the map and thus do not scale efficiently.

Some approaches have been proposed before, such as using an auction-based system3 and using Voronoi maps4. However, these methods can only work on specific environments and cannot be generalized.



Voronoi based approaches restricts robots from moving into other areas, which may cause problems [5]

A better approach is to treat this problem as a special case of the Multiple Travelling Salesman Problem (mTSP). The (mTSP) problem is an extension of the famous NP-Hard Travelling Salesman problem. In this case, instead of having one agent traverse the entire grid, we have multiple agents. Our task is to minimize the total time taken to solve the entire problem.

Unfortunately, just like the TSP problem, the multi-agent TSP problem can only be ideally solved with non-polynomial time complexity. This means that for us to make it scalable we must use a heuristic to get an acceptable solution.

Another real-life issue we face is that not all tasks have the same urgency. Hence, we must also find

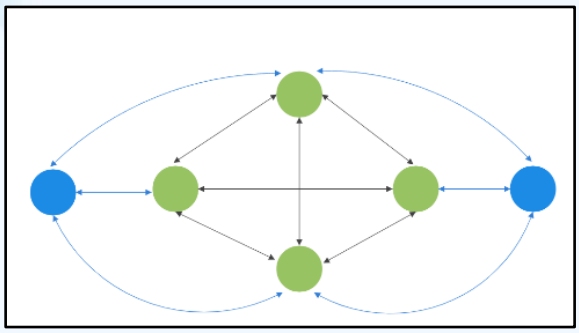
[3] https://www.researchgate.net/publication/264873563_Auctions_for_task_allocation_to_robots

[4] https://www.researchgate.net/publication/224557569_Local_Voronoi_Decomposition_for_multi-agent_task_allocation

[5] https://ieeexplore.ieee.org/document/9465629

a way to factor in task priorities while making our calculations.

Taking all this in mind, our solution tackles this problem using a greedy mTSP approach with ML augmented weights.

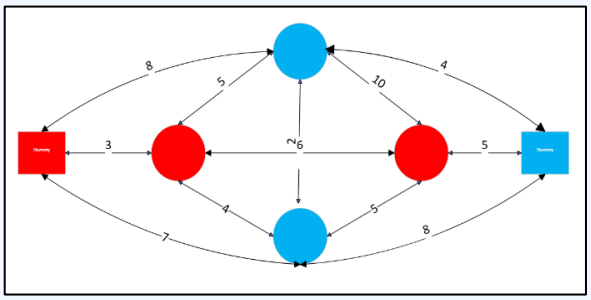


An example of the mTSP problem. Here the blue nodes are the starting depots for our agents.

Algorithmic Overview

An overview of our algorithm is as follows.

- We have a set of Tasks along with a set of robots. There can be any number of tasks and robots.
- We generate a graph by creating a node for every task. We also compute the edge weights between each pair of nodes, based on the distance and task urgency using a neural network.
- When task allocation starts, we create a "dummy" node for each agent we have. These dummy nodes are can only travel to one of the given tasks and are deleted after the task allocation process.
- We set the robot position to the corresponding dummy node.
- We then find the robot-task pair which has the smallest edge value. We consider this to be the most optimal action. We accordingly update that robot's position to the best node.
- We continue this process till all tasks have been assigned.



The allocation of tasks according to the algorithm. The total distance lengths of the bots are 9 and 6. The order in which the edges are selected are 3,4,2,6.

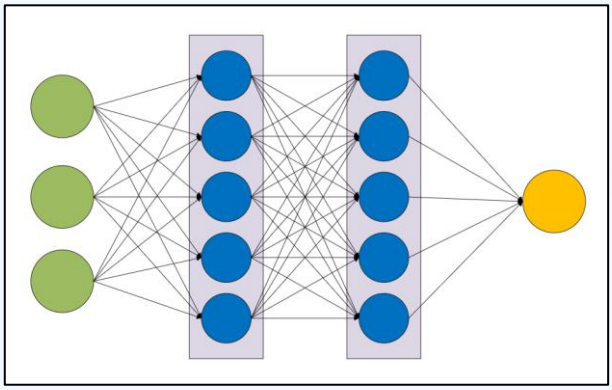
Since this algorithm uses greedy approach based on a heuristic, we do not have to deal with a polynomial time complexity.

Computing the Heuristic

To ensure that our greedy algorithm comes up with a reasonably good solution, we have used a ML based approach to calculate our edge heuristics. We have 3 inputs which we need to consider: -

1. Distance between two nodes.
2. The Robot's backlog (tasks already assigned to it in terms of distance)
3. Urgency of the task

To calculate this, we use an artificial neural network with 2 hidden layers of 5 neurons each. The first hidden layer has a sigmoid activation function while the second uses RELU.



A diagram showing our neural network.

To train the network, we have experimentally found some optimal weights for some discrete input vectors. To generalize this for continuous values, we train our network using these discrete data points till we reach convergence.

While traversing the graph to allocate tasks, the value of the edge is given by.

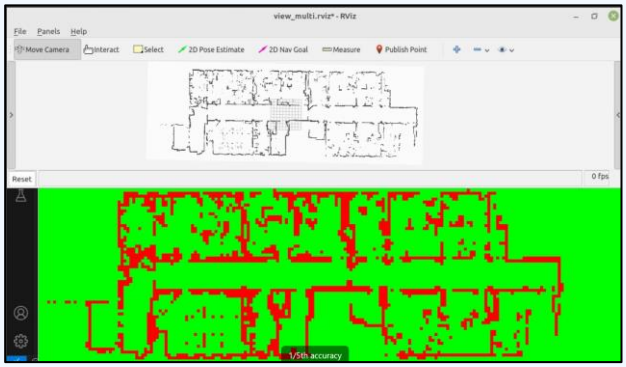
$$edge = \frac{distance}{weight}$$

Since the neural network considers the effect of both backlog of the robot and priority of the task, we do not need to manually create an algorithm to include them in our calculations.

To implement all this, we have written our own code in python. Further implementation details can be found in the submission.

Map Pooling

We have used the A* algorithm to find the distance between the various task and dummy nodes. This leads to problems on large maps, since running A* algorithm multiple times on a large map is time consuming.

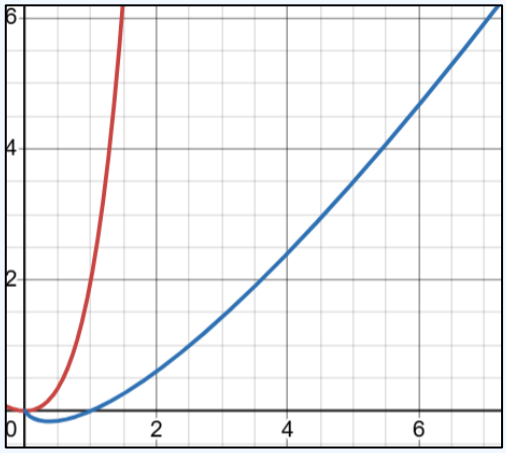


Comparison between original map and pooled map

To decrease running time, we have condensed our map using a pooling layer. Since we only care about the relative distances, this allows us to make faster calculations without affecting accuracy in any way

Performance Analysis

Trying out every combination has a non-polynomial time complexity. Thus, this approach will scale very poorly, with large waiting times for even small number of tasks. Our optimization however runs in polynomial time complexity and can thus easily handle large number of tasks. We have also applied a pooling layer to decrease the map size for calculations, leading to even faster computations

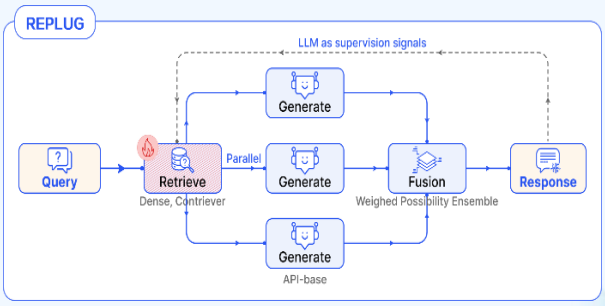


Time complexity analysis of Task Allocation
The red line is if we try to solve the problem normally
The blue line is our greedy heuristic based approach

LLM INTEGRATION

To add a layer of complexity to user communication, an LLM (Large Language Model) based interface has been incorporated in the project which can both take direct (x,y) inputs as well as human written commands.

The current architecture being used is a complex version of a RAG (Retrieval Augmented Generation) based chatbot, specifically a Modular Agentic RAG with Replug Flow. For the simple task of custom object detection, only a few modules have been used, but the support for many more submodules and agents is inbuilt into the architecture for further updates.



The overall structure of the LLM used[6]

[6] https://arxiv.org/html/2407.21059v1#bib.bib35

The Flow of the RAG is as follows:

- Embed the raw text file with the item names and destinations into a lower dimension vector space using FAISS vectorstore and Google GenAI embedder.
- Take the input of the user, embed it into the same vector space, and gather the k nearest neighbour document chunks. (Here k=1)
- Pass this query to a QueryRouter agent which decides whether the input query is a command or a question regarding an item.
- Pass the input of the user as the query and these documents as the context to the Generator Agent which injects these into the LLM and finally returns the answer.
- Then, if the answer is not of a valid format, the query is plugged back into the flow once again until the termination condition is triggered.

This chatbot is restricted for the use of this specific program and an attempt to use this LLM as a general chatbot will result in errors The main aim of optimizing the chatbot is to reduce hallucinations and minimize latency, hence a simple and effective Replug Flow has also been implemented.

The LLM currently being used is the Gemini-1.5-Flash, boasting billions of parameters and low latency. It is also free to use.

Using the LLM

To make it easy to use the LLM, we have designed our own custom interface using python.

The chatbot takes a task such as:

```
"Go to the TV"
```

And converts it into this format:

```
Item:"Television"
Destination: (200,100)
Urgency: 5
```

This can then be directly fed into the task allocation system.

Moreover, a simpler method of directly entering the (x,y) positions has also been implemented.

Directly entering the (x,y) co-ordinates will inject this as a simple task to the robot allocation system and avoid the latency period of the LLM completely, leading to even faster allocation.

SCALABILITY ANALYSIS

Any swarm robotics framework must be scalable to account for multiple robots and tasks. In this section we briefly describe the features in our solution which allow this while also giving instructions on necessary changes to the source code to accommodate more robots.

- The map merge package has been designed in such a way so that it can accommodate as many robots as required. It searches for all ROS topics which end with */map* and use that data to generate the final image.
- The task allocation system has been designed to handle as many tasks and robots as possible. Since each robot and task is a node in the graph, it is very easy to scale it.
- Our neural network is pre-trained, and hence can compute the weights for each edge quickly.

Overall, by ensuring that our algorithms do not have an exponential rate of growth, we have made our entire project scalable even for multiple robots

CHALLENGES FACED

Facing problems is inevitable while solving any real-life problem. In this section we discuss some of the challenges we faced and how we overcame them.

- **Lack of an efficient Task allocation system**
  There are very few good ROS packages existing which solve the problem to task allocation efficiently. There are some market based "auction" systems[7] which exist. These however cannot efficiently give the order in which tasks should be performed

  Solution:
  We decided to write our own task allocation algorithm, based on solutions of the mTSP problem.

- **User Friendly Interface**
  Most ROS packages are built for use by developers. This makes it difficult for non-technical people to use these powerful tools.

  Solution:
  We added an LLM powered GUI in which people only have to add commands. These commands are then parsed and fed as instructions to the remaining scripts.

- **Efficient pathfinding algorithms**

  Traditional pathfinding algorithms such as A*, BFS or DFS are unable to account for dynamic obstacles and must recalculate the entire path every time they detect a new obstacle.

  Solution:-
  We instead use the D* family of algorithms to efficiently recompute weights for dynamic obstacles. These are fast algorithms which outperform traditional methods for non-static environments.

- **Factoring in task priority while allocating tasks**
  In a real-life system, not all tasks will have the same priority. Therefore, we should take into consideration how important a particular task is. Tasks with lower urgencies should be done later while more pressing tasks should be done first

  Solution:-
  We use a ML model built using pytorch and trained on experimentally found weights. This network is then used in the Task Allocation algorithm to modify the edge weights.

- **Large Complex maps hinder scalability**
  Large maps store more data, and hence doing calculations on them take more time. This becomes more evident when we have many robots and tasks as well.

  Solution:
  We have created a pooling layer to compress map size while preserving the general obstacle data. This allows faster calculations.

- **Providing direct points to the LLM leads to hallucinations**
  Gemini-flash being a low parameter model cannot differentiate between points and items and ends up combining their co-ordinates instead.

  Solution:
  To solve this problem, we have implemented a bi-pronged approach of dividing the user input into direct commands and written commands, tackling each case independently, meanwhile still allowing for seamless integration between both the cases.

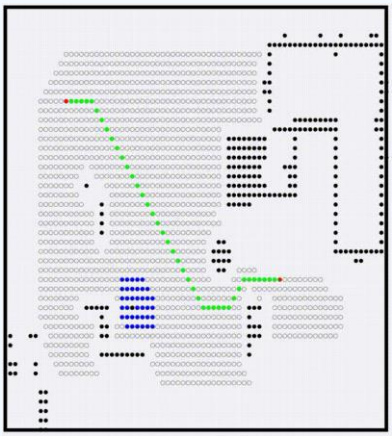
[7] http://wiki.ros.org/task_allocation

FUTURE SCOPE AND IMPROVEMENTS

There are many areas in which more work can be done on our proposed system to make it more efficient. Some areas of further improvements are

- **IMPROVED WEIGHT HEURISTIC**
  The current weight prediction heuristic can only look as far as one task ahead. A better model should consider more information about the environment.

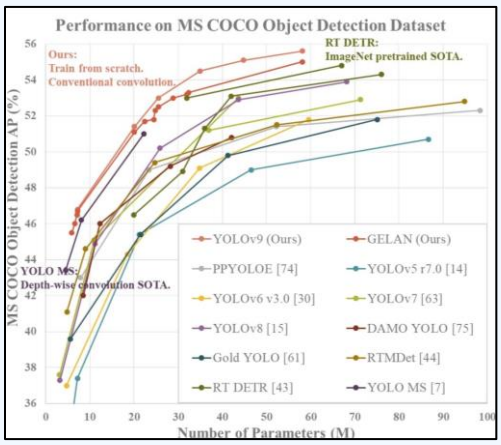
- **BETTER PATHFINDING ALGORITHMS**
  Pathfinding algorithms such as D* lite are better suited to dynamic environments and can often have faster runtime speeds.



D* Lite in Action

- **USING BETTER LLMs**
  Free versions of LLMs are prone to hallucinations and can return incorrect data. To overcome this once should use paid LLMs such as ChatGPT. Such an LLM can also interpret more complicated tasks

- **BETTER IMAGE DETECTION Models**
  Better YOLO models such as YOLOv9 have faster detection capabilities and are more accurate. These can be used in real-life environments where greater precision is required.

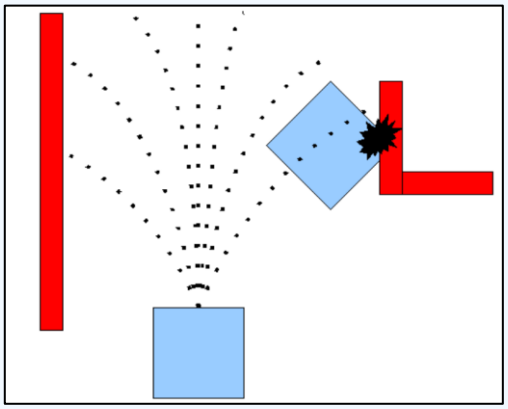


Comparison of different models[8]

- **ADDING CUSTOM IMAGES TO DETECTION MODEL**
  Currently our YOLOv5 model can only detect the classes defined in the base model. We can add custom images as well to make it more suitable for real-life use.

- **INTEGRATING LOCAL PLANNERS**
  While global planners such as D* and D* lite can update with changes in environment, we can also use local planners such as DWA (Dynamic Window Approach) to fine tune our velocity commands. This will make robot movement much smoother.

  To implement this, one can use the ROS package dwa_local_planner[9]

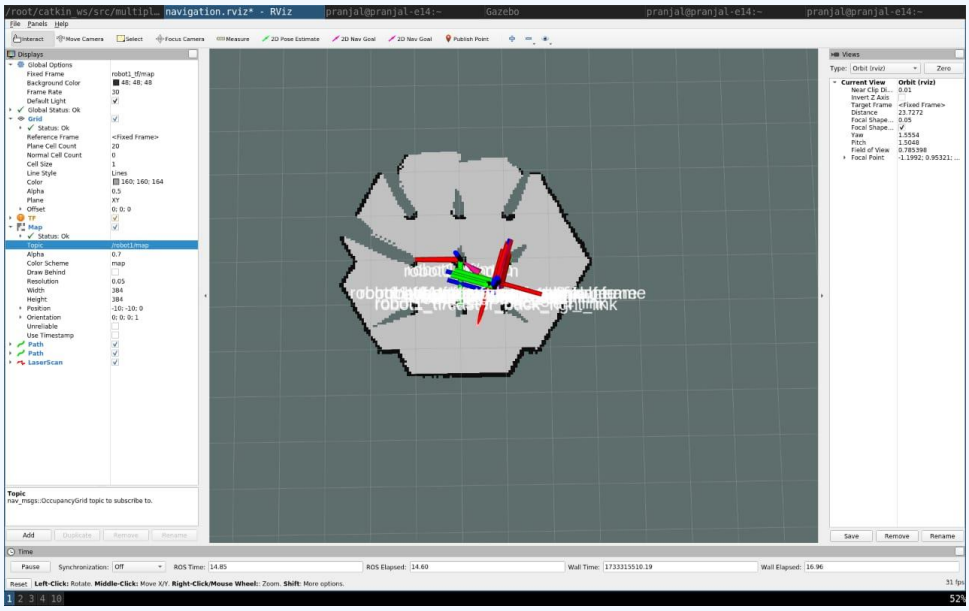


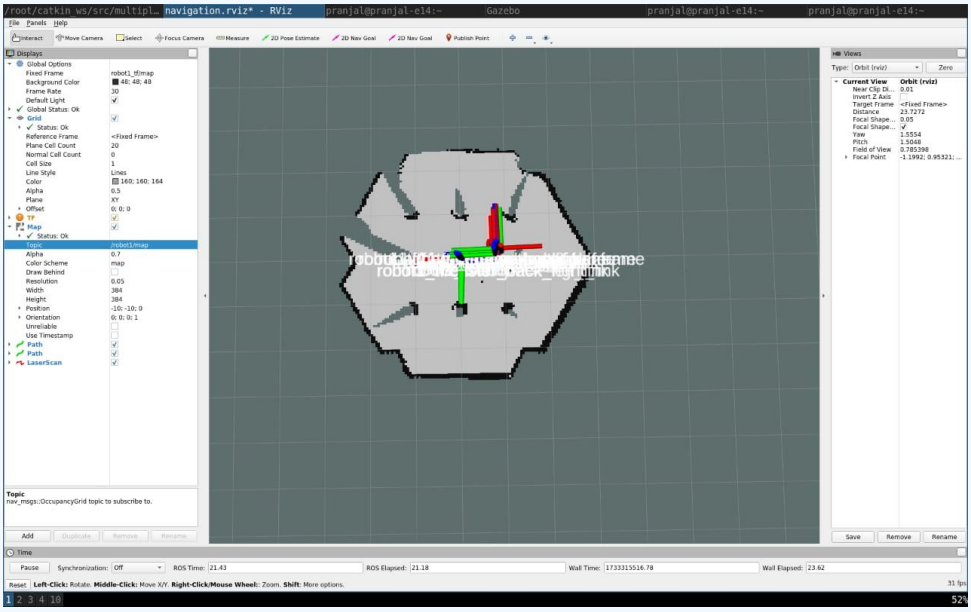
Using DWA to fine tune our movements[10]

[8] https://viso.ai/computer-vision/yolov9/
[9] http://wiki.ros.org/dwa_local_planner
[10] http://wiki.ros.org/dwa_local_planner

APPENDIX

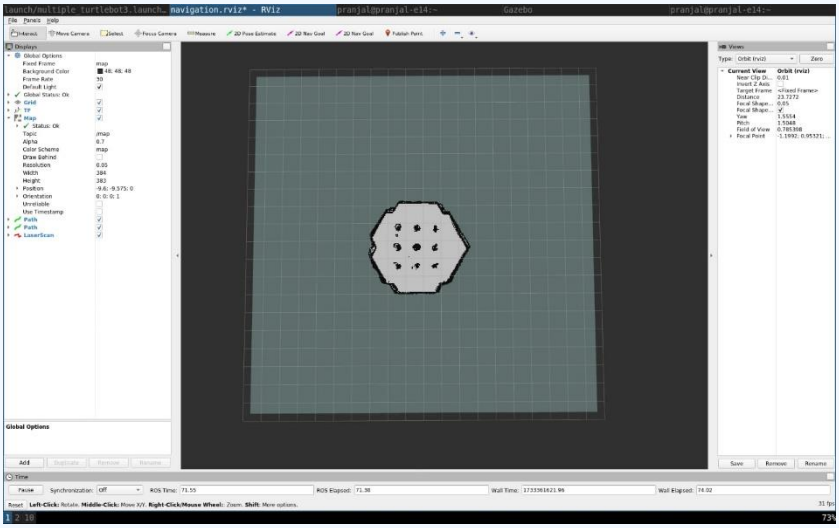
FRONTIER EXPLORATION





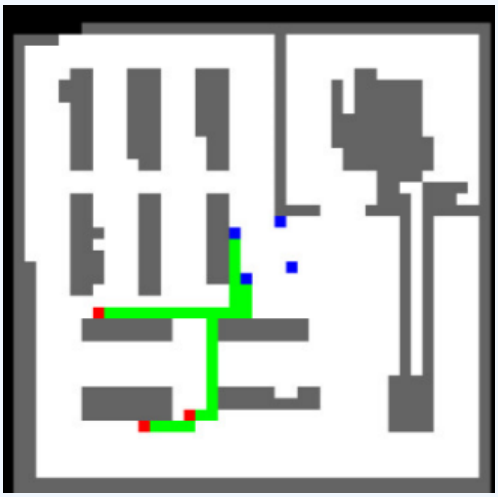
Frontier exploration in progress

MAP MERGE

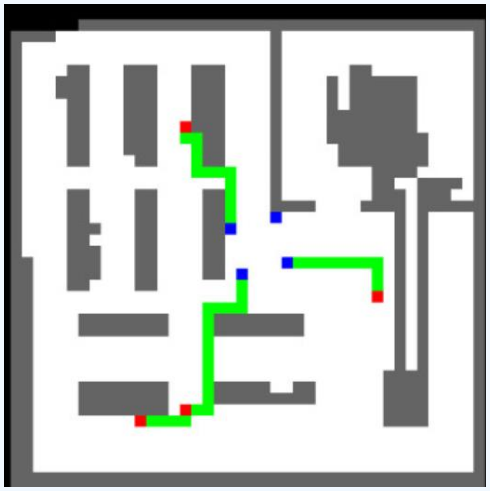


Final image after map

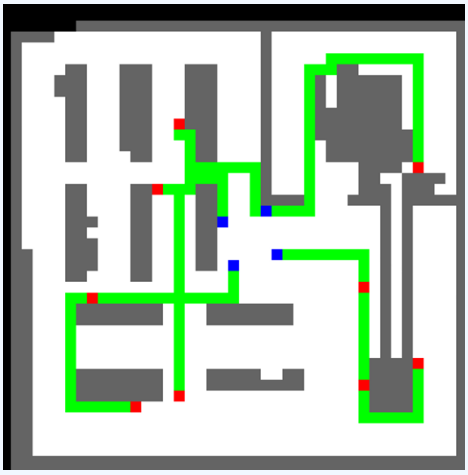
TASK ALLOCATION



Task Allocation for 2 tasks



For 3 tasks

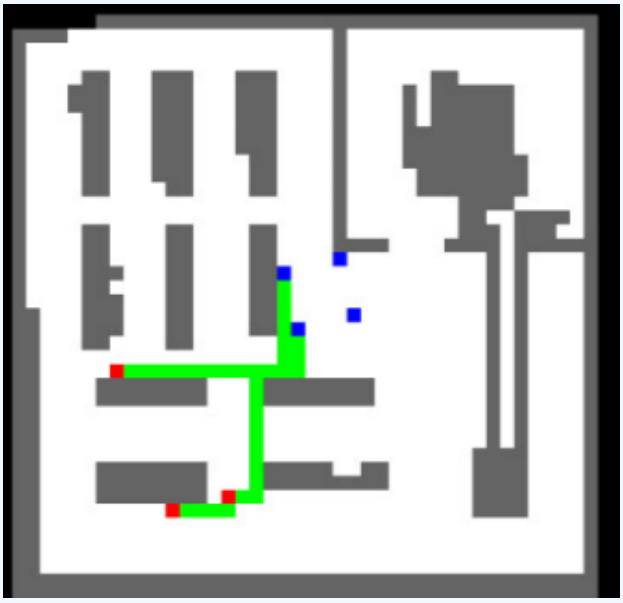


For 9 tasks

LLM SIMULATION (COMMAND LINE)

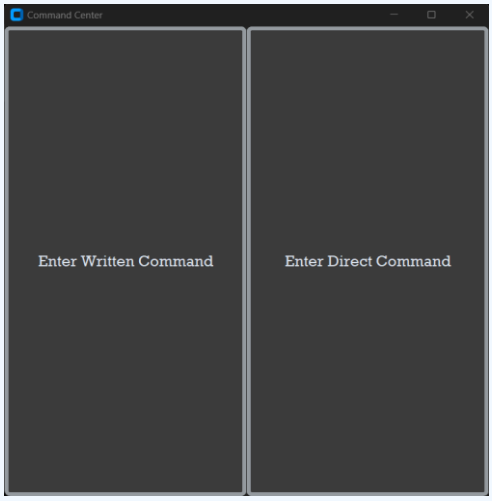
```
re/path_planner/path_planner/src/Task_Allocation$ python3.10 LLM.py
WELCOME TO THE COMMAND TERMINAL
ENTER ALL CO-ORDINATE TASKS IN (X,Y,Priority) FORMAT (PRINT EXIT WHEN DONE)
(-7,3,2.4)
exit
ENTER ALL ITEM TASKS (Example:- Go to fire extinguisher)
Enter Command: go to fire extinguisher urgently
Processing Query : go to fire extinguisher urgently
Documents Retreived
(-3, 7.2)
10
Enter Command: go to painting later
Processing Query : go to painting later
Documents Retreived
(-5, 8)
0
Enter Command:
```

Example Terminal



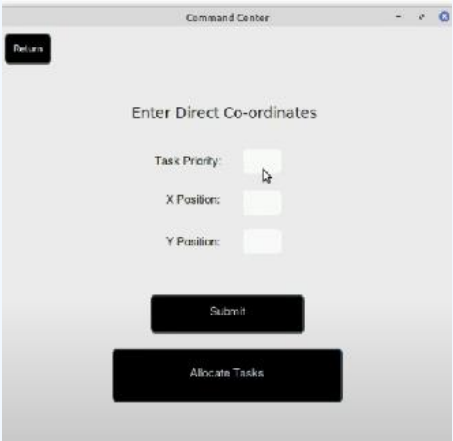
Corresponding Task Distribution

LLM INTERFACE



Main Page for the interface

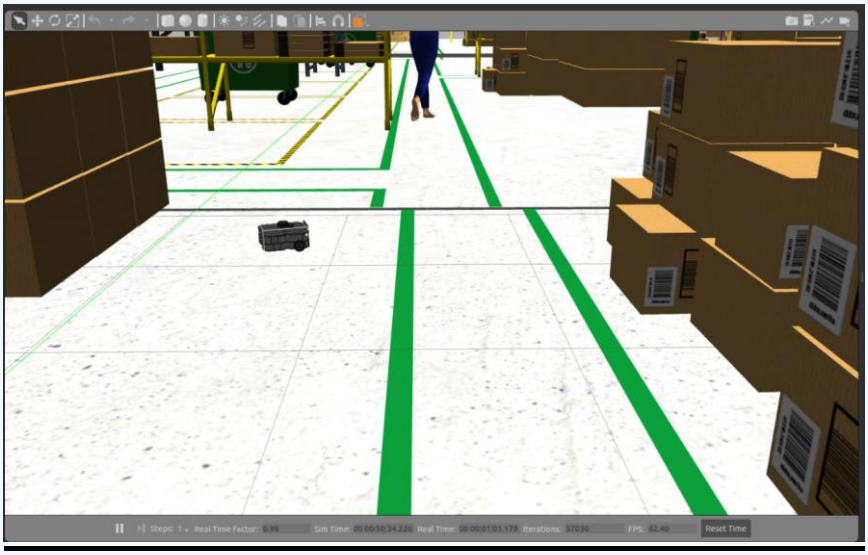
For sending instructions to the LLM

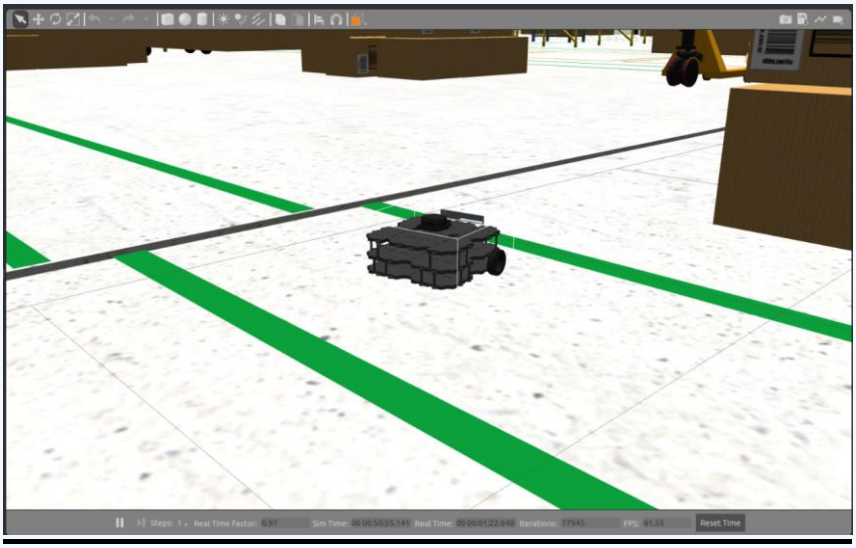


For giving direct co-ordinates

V

GAZEBO SIMULATIONS





Robots performing dynamic pathfinding on Gazebo